GEEK Streaming Support Assistant

Portfolio Case Study | From Hand-Mapped Logic to a Branded No-Code Support Experience

Developer: Evangela L.
Role: Conversation design, Python prototyping, no-code development, testing, and documentation
Tools: Google Colab (Python), Whimsical, Landbot, and Google Sheets
Project stage: Published and fully validated proof of concept

Validation Complete

> The planning, build, formal testing, defect correction, retesting, screenshots, and screen-recorded demonstration are complete. The final validation package covers 108 checks across the Python prototype, flowcharts, and published Landbot experience.

1. Project at a Glance

GEEK Streaming Support Assistant is a guided troubleshooting chatbot for everyday users dealing with common streaming-device problems. Instead of presenting a long support article or a generic contact form, the assistant asks focused questions, recommends a practical first-line fix, and checks whether the problem was resolved. When more help is needed, the current Landbot prototype collects the information required to prepare the case for escalation.

The Problem

Streaming-device support can become frustrating quickly. Users may not know whether a blank screen points to the television input, an HDMI connection, the streaming device, or something else. Traditional support pages often list every possible fix at once, which can leave less technical users guessing about what applies to them. I wanted to create a calmer experience that narrows the problem one decision at a time.

Target Audience

The assistant is designed for people who want clear self-service help without heavy technical language. The wording is short, direct, and supportive, and the button-based format reduces the chance of users getting stuck because they entered an unexpected response.

Project Goals

- Reduce customer frustration during common streaming-device issues
- Route users to the correct solution faster
- Demonstrate conversational flow design and troubleshooting logic
- Showcase no-code development and decision-tree architecture

2. The Solution

The solution is a published, button-driven Landbot prototype built from logic first mapped by hand, then modeled in Python and translated into visual flowcharts. It supports four troubleshooting categories: Power, Connection, Audio/Video, and Remote.

How the Experience Works

- The user chooses one of four support categories.
- The assistant narrows the category to one of two symptoms.
- A focused diagnostic question determines which troubleshooting step appears.
- The user confirms whether the recommended action worked.
- A resolved issue receives a success message and an option to get more help.
- An unresolved issue moves to a support-request form that collects contact and device details.
- The user receives a personalized confirmation after submitting the form.

Logic Scope

The Python prototype contains eight symptom routes. Each symptom can receive two diagnostic answers, and each troubleshooting action can end as resolved or unresolved. That creates 32 valid end-to-end paths: eight symptoms × two diagnostic answers × two final outcomes. The main menu looks simple, but the branching underneath it required careful planning and testing.

3. Development Process

Phase 1: Hand-Mapped Planning

I began by writing the conversation paths in a physical notebook. Slowing down at this stage helped me see how each user choice should connect before I opened any software. It also gave me a roadmap that I could compare against the later versions.

Phase 2: Python Logic Prototype

I built the first working version in Google Colab using Python if/elif/else statements. This version modeled the four categories, eight symptom routes, diagnostic answers, troubleshooting instructions, success outcomes, escalation concepts, and fallback behavior. The code served as a functional blueprint for the conversation.

Phase 3: Visual Flowcharts

I translated the logic into flowcharts so I could trace the branches visually. The charts made the relationships between symptoms, diagnostic questions, fixes, and outcomes easier to review. They also gave me a bridge between the Python prototype and the no-code build.

Phase 4: Landbot Implementation

I rebuilt the core logic in Landbot and expanded it into a more complete user experience. Text inputs became buttons, shared outcomes became reusable paths, and the assistant gained navigation, form collection, personalization, and visual branding. This stage turned the logic prototype into something another person could open and use.

4. What Changed During Implementation

The final Landbot build is intentionally more developed than the original Python logic and flowcharts. I kept the earlier artifacts as evidence of the starting point instead of editing them to look as though every feature existed from the beginning.

- Added a “Do you need help with another issue?” step after completed routes
- Connected continued-help responses back to the main menu
- Created one shared support-request form for unresolved issues
- Added required fields, email validation, contact permission, saved responses, and a personalized confirmation

5. Design and Brand Decisions

I wanted the assistant to feel helpful without looking cold or overly technical. The earthy navy, olive, and cream palette reflects my personal design style while keeping the interface calm and readable. The prompts are brief, the choices are clear, and the repeated interaction pattern helps users understand what will happen next. The final design supports the same idea behind GEEK: tech help without the headache.

6. Testing and Validation

Testing was treated as a documented project phase rather than an invisible final check. I used one workbook with separate tabs for the Python logic, flowchart structure, published Landbot experience, and defect-and-retest history.

Completed Test Results

- Python Logic Tests: 40 executed. The initial run produced 37 passes and three failures. After two code defects were corrected, all three failed cases passed retesting.
- Flowchart Validation: 28 of 28 checks passed, covering category completeness, branch accuracy, labels, connectors, and endpoints.
- Landbot User Acceptance Testing: 40 of 40 tests passed across core routing, navigation, forms, validation, personalization, and published-session behavior.
- Defect and Retest Log: two Python defects were recorded, corrected, and successfully retested; no open defects remain.

Final validation result: 108 of 108 planned checks passed after corrections and retesting. The project makes no customer-adoption or business-impact claims because it remains a portfolio proof of concept.

Captured Visual Evidence

The completed screen recording shows a representative success route, a return to the main menu, an unresolved route, form validation, form submission, personalized confirmation, and the final closing experience. Seven selected screenshots capture the main menu, troubleshooting step, success message, return navigation, support form, validation behavior, and personalized final confirmation.

7. Challenges and How I Worked Through Them

Planning for Real User Behavior

My early design focused on the expected troubleshooting path. Formal testing exposed three cases involving extra spaces, capitalization, and an invalid yes/no response. I added input normalization and a reusable validation loop, then reran those cases until they passed. That made the Python prototype more realistic and showed why fallback behavior has to be tested, not assumed.

Managing Connections in Landbot

Some of the most time-consuming problems came from connections rather than the troubleshooting content itself. Similar block names, Jump To selections, saved fields, checkbox values, and form connections could send the user to the wrong place or prevent information from appearing correctly. I worked through those problems by slowing down, renaming similar items, tracing each connection, and testing the route again.

Keeping the Project Claims Accurate

The original Python logic and flowcharts used messages such as “Create Ticket” to represent the intended escalation destination. The current Landbot version collects and prepares support details, but it is not connected to a real ticketing system or live-agent queue. I kept that difference visible because it shows progression and sets a clear direction for the next version.

8. Verified Results

The project now includes a working published prototype, completed validation, corrected defects, and a complete set of supporting artifacts. These are verified build and testing outcomes, not claims about customer adoption or business impact.

- Four troubleshooting categories and eight symptom routes
- Thirty-two valid end-to-end paths in the Python prototype
- A visual flowchart set that maps the original decision structure
- A branded Landbot chatbot with reusable navigation and shared outcomes
- A support-request form with saved fields and personalized confirmation
- A completed workbook covering 108 Python, flowchart, and Landbot checks
- Two Python defects identified, corrected, and successfully retested

9. Current Boundaries

- The chatbot is a proof of concept, not a production support system.
- The form does not yet create a ticket in an external platform.
- There is no automatic live-agent transfer.
- The experience uses structured buttons rather than natural-language issue detection.
- Support analytics and recurring-issue reporting are not yet connected.
- The troubleshooting scope is limited to the eight symptoms included in this version.

10. Future Version

- Connect the escalation form to Zapier or Make for workflow automation
- Send support requests to Google Sheets, email, or a ticketing platform
- Add a real ticket number and status confirmation
- Create a live-agent or Tier 2 handoff
- Add natural-language intent detection while keeping button choices available
- Track recurring issues and unsuccessful troubleshooting routes
- Expand device-specific support and add more symptom branches

11. Skills Demonstrated

- Conditional logic and decision-tree design
- Python prototyping with if/elif/else statements
- Conversation design and user-centered troubleshooting
- No-code development in Landbot
- Flowchart mapping and branch validation
- Form configuration, saved variables, and personalization
- Structured QA planning, defect tracking, and retesting
- Scope control and accurate technical documentation
- Visual branding and interface consistency

12. Personal Reflection

What worked best during this project was slowing down and planning before building. Writing the logic by hand helped me visualize the user's journey and gave me something concrete to follow when I moved into Python, flowcharts, and Landbot. The project also reminded me that a simple interface can hide a lot of decision-making underneath it.

I enjoyed the point where logic, design, and problem-solving came together. I did more than move users through a set of questions. I planned a support experience, built it in stages, found gaps, corrected connections, and created a testing process to verify the final result. The finished Landbot feels like my work: earthy, simple, clear, and helpful without feeling overly technical.

13. Supporting Portfolio Evidence

The completed portfolio package includes:

- Polished project documentation: GEEK Streaming Support Assistant | Project Documentation
- Google Colab Python notebook and final input-validation fix: StreamingChatBotAI-Project.ipynb
- Project visuals and screenshots in the AI Project folder: AI Project
- Completed testing and validation workbook: GEEK Streaming Support - Testing & Validation Workbook
- Seven selected screenshots from the published chatbot
- Screen-recorded demonstration of the representative end-to-end experience: GEEK Streaming Support.mp4
- Published chatbot link ready for the portfolio page
- Final testing summary: 108 checks passed after two defects were corrected and retested